Problem Set 7

1. (7 points) Consider an n-node complete binary tree T, where $n = 2^d - 1$ for some d. Each node v of T is labeled with a real number x_v. You may assume that the real numbers labeling the nodes are all distinct. A node v of T is a local minimum if the label x_v is less than the label x_w for all nodes w that are joined to v by an edge.

   You are given such a complete binary tree T, but the labeling is only specified in the following implicit way: for each node v, you can determine the value x_v by probing the node v. Show how to find a local minimum of T using only $O(\log n)$ probes to the nodes of T.

2. (8 points) (Exercise 6.6 from the textbook) In a word processor, the goal of "pretty-printing" is to take text with a ragged right margin, like this,

   Call me Ishmael.
   Some years ago,
   never mind how long precisely,
   having little or no money in my purse,
   and nothing particular to interest me on shore,
   I thought I would sail about a little
   and see the watery part of the world.

   and turn it into text whose right margin is as "even" as possible, like this.

   Call me Ishmael. Some years ago, never
   mind how long precisely, having little
   or no money in my purse, and nothing
   particular to interest me on shore, I
   thought I would sail about a little
   and see the watery part of the world.

   To make this precise enough for us to start thinking about how to write a pretty-printer for text, we need to figure out what it means for the right margins to be "even". So suppose our text consists of a sequence of words, $W = \{w_1, w_2, ..., w_n\}$, where w_i consists of c_i characters. We have a maximum line length of L. We will assume we have a fixed-width font and ignore issues of punctuation or hyphenation.

   A *formatting* of W consists of a partition of the words in W into lines. In the words assigned to a single line, there should be a space after each word except the last; and so if $w_j, w_{j+1}, ..., w_k$ are assigned to one line, then we should have

   $$\left[\sum_{i=j}^{k-1}(c_i + 1)\right] + c_k \leq L.$$

   We will call an assignment of words to a line *valid* if it satisfies this inequality. The difference between the left-hand side and the right-hand side will be called the *slack* of the line–that is, the number of spaces left at the right margin.

Give an efficient algorithm to find a partition of a set of words W into valid lines, so that the sum of the squares of the slacks of all lines (including the last line) is minimized.

3. (7 points) A large collection of mobile wireless devices can naturally form a network in which the devices are the nodes, and two devices x and y are connected by an edge if they are able to directly communicate with each other (e.g., by a short-range radio link). Such a network of wireless devices is a highly dynamic object, in which edges can appear and disappear over time as the devices move around. For instance, an edge (x, y) might disappear as x and y move far apart from each other and lose the ability to communicate directly.

   In a network that changes over time, it is natural to look for efficient ways of maintaining a path between certain designated nodes. There are two opposing concerns in maintaining such a path: we want paths that are short, but we also do not want to have to change the path frequently as the network structure changes. (That is, we'd like a single path to continue working, if possible, even as the network gains and loses edges.) Here is a way we might model this problem.

   Suppose we have a set of mobile nodes V, and at a particular point in time there is a set E_0 of edges among these nodes. As the nodes move, the set of edges changes from E_0 to E_1, then to E_2, then to E_3, and so on, to an edge set E_b. For $i = 0, 1, 2, ..., b$, let G_i denote the graph (V, E_i). So if we were to watch the structure of the network on the nodes V as a "time lapse," it would look precisely like the sequence of graphs $G_0, G_1, G_2, ..., G_{b-1}, G_b$. We will assume that each of these graphs G_i is connected. Now consider two particular nodes $s, t \in V$. For an $s - t$ path P in one of the graphs G_i, we define the length of P to be simply the number of edges in P, and we denote this $\ell(P)$. Our goal is to produce a sequence of paths $P_0, P_1, ..., P_b$ so that for each i, P_i is an $s - t$ path in G_i. We want the paths to be relatively short. We also do not want there to be too many changes—points at which the identity of the path switches. Formally, we define $changes(P_0, P_1, ..., P_b)$ to be the number of indices i $(0 \leqslant i \leqslant b - 1)$ for which $P_i \neq P_{i+1}$.

   Fix a constant $K > 0$. We define the cost of the sequence of paths $P_0, P_1, ..., P_b$ to be

$$cost(P_0, P_1, ..., P_b) = \sum_{i=0}^{b} \ell(P_i) + K \cdot changes(P_0, P_1, ..., P_b).$$

   (a) Suppose it is possible to choose a single path P that is an $s - t$ path in each of the graphs $G_0, G_1, ..., G_b$. Give a polynomial-time algorithm to find the shortest such path.

   (b) Give a polynomial-time algorithm to find a sequence of paths $P_0, P_1, ..., P_b$ of minimum cost, where P_i is an s-t path in G_i for $i = 0, 1, ..., b$.

4. (8 points) On most clear days, a group of your friends in the Astronomy Department gets together to plan out the astronomical events they're going to try observing that night. We'll make the following assumptions about the events.

   - There are n events, which for simplicity we'll assume occur in sequence separated by exactly one minute each. Thus event j occurs at minute j; if they don't observe this event at exactly minute j, then they miss out on it.

- The sky is mapped according to a one-dimensional coordinate system (measured in degrees from some central baseline); event j will be taking place at coordinate d_j, for some integer value d_j. The telescope starts at coordinate 0 at minute 0.
- The last event, n, is much more important than the others; so it is required that they observe event n.

The Astronomy Department operates a large telescope that can be used for viewing these events. Because it is such a complex instrument, it can only move at a rate of one degree per minute. Thus they do not expect to be able to observe all n events; they just want to observe as many as possible, limited by the operation of the telescope and the requirement that event n must be observed.

We say that a subset S of the events is viewable if it is possible to observe each event $j \in S$ at its appointed time j, and the telescope has adequate time (moving at its maximum of one degree per minute) to move between consecutive events in S.

The problem. Given the coordinates of each of the n events, find a viewable subset of maximum size, subject to the requirement that it should contain event n. Such a solution will be called optimal.

Example. Suppose the one-dimensional coordinates of the events are as shown here.

Event	1	2	3	4	5	6	7	8	9
Coordinate	1	-4	-1	4	5	-4	6	7	-2

Then the optimal solution is to observe events 1, 3, 6, 9. Note that the telescope has time to move from one event in this set to the next, even moving at one degree per minute.

(a) Show that the following algorithm does not correctly solve this problem, by giving an instance on which it does not return the correct answer.

Mark all events j with $|d_n - d_j| > n - j$ as illegal (as observing them would prevent you from observing event n)
Mark all other events as legal
Initialize current position to coordinate 0 at minute 0
while not at end of event sequence **do**
  Find the earliest legal event j that can be reached without exceeding the maximum movement rate of the telescope
  Add j to the set S
  Update current position to be coord. d_j at minute j
end while
Output the set S

... In your example, say what the correct answer is and also what the algorithm above finds.

(b) Give an efficient algorithm that takes values for the coordinates $d_1, d_2, ..., d_n$ of the events and returns the size of an optimal solution.